计算机组成原理（2026 年）
简介
前言
概览
实验1 汇编语言与监控程序
硬件平台模拟环境
监控程序和终端程序
RISC-V 指令概况
RISC-V 汇编语言程序的调试
实验内容
实验2 Vivado 实验之 ALU 和寄存器堆实验
SystemVerilog 语言
通过例子学硬件描述语言
总述
加法器
按钮开关
秒表
计数器
无状态仲裁器
循环优先级仲裁器
代码规范
仿真
Vivado 使用入门

目录

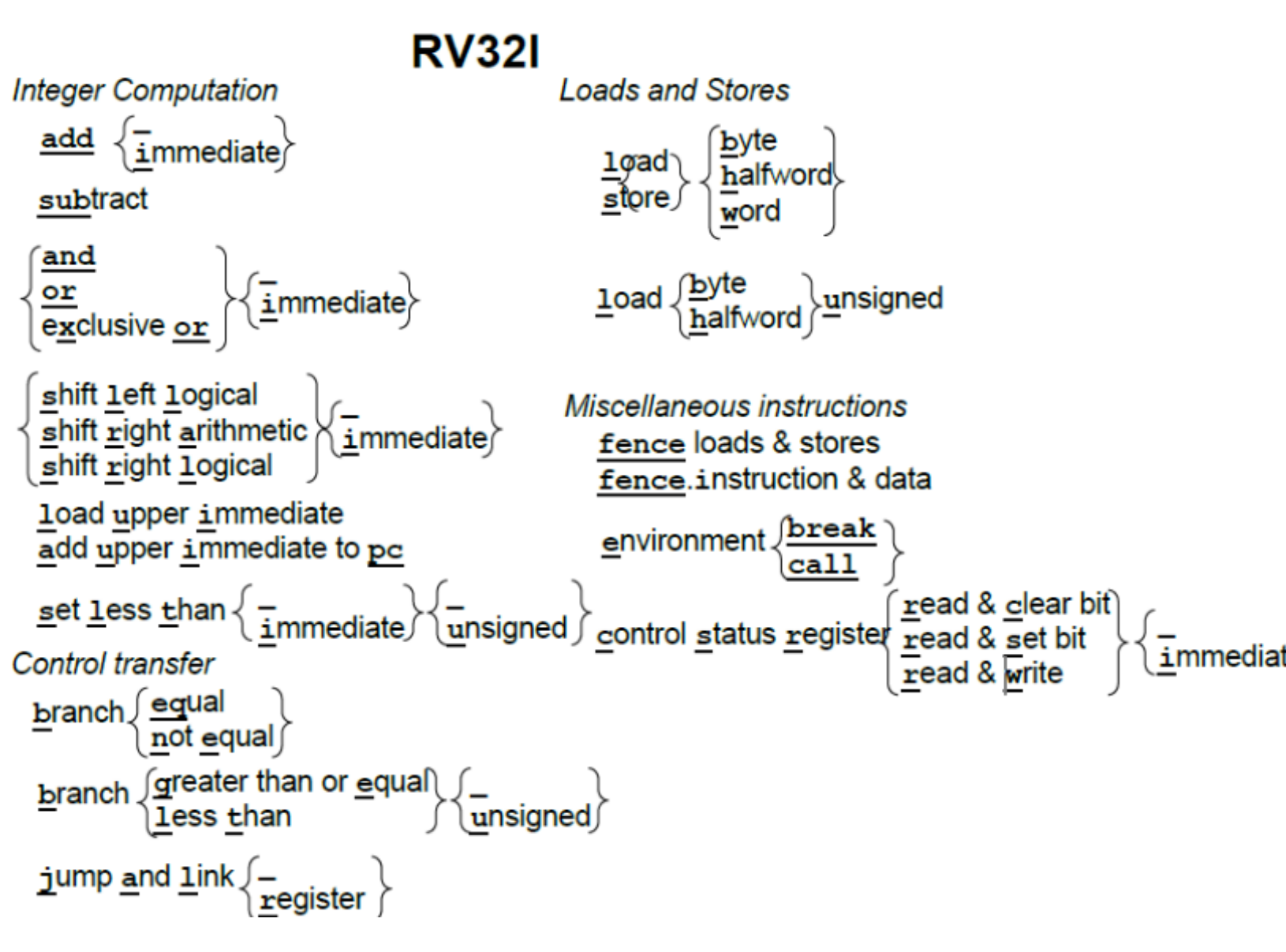
模块化的指令体系结构设计
最基本的 RV32I 指令集
寄存器
指令格式概述
整数运算指令
访存指令
条件分支指令
大位宽数据加法
检测加法溢出
无条件跳转指令
杂项指令
RV32I 函数调用惯例
伪指令
汇编器指导语句

RISC-V 指令概况

模块化的指令体系结构设计

RISC-V 的指令集体系结构是模块化的。最基础的指令是 RV32I，即 32 位的整数指令集，包含了所有的 RISC-V 处理器都必须要实现的指令。模块化的方式让 RISC-V 的指令集体系结构有了在基础指令集上进行扩展的能力：RV64I，这是 64 位的整数指令扩展；RV32M 是乘法指令扩展；RV32F 是单精度浮点扩展；RV32D 是双精度浮点扩展等。RV32IMFD 就代表了把对应的乘法，单精度，双精度模块扩展到基础的 RV32I 中。

最基本的 RV32I 指令集



指令分为四大类，带下划线的字母从左到右连接就组成了 RV32I 指令名称，花括号{}表示里面的每项都是指令的不同变体，上划线表示不包含这个字母的也是一个指令名称，例如，and 这一组指令表示以下六个指令：and，or，xor，andi，ori，xori。

寄存器

RV32I 指令集体系结构中一共有 32 个寄存器，x0 到 x31，其中 x0 总是 0。

与其它的指令集体系结构一样，RV32I 中的一些寄存器也有别名，别名可以帮助记忆关于调用惯例方面的规范。

下面是寄存器及其别名的对应关系：

```
x0 / zero
x1 / ra (return address)
x2 / sp (stack pointer)
x3 / gp (global pointer)
x4 / tp (thread pointer)
x5 / t0 (temporary)
x6 / t1
x7 / t2
x8 / s0 / fp (saved register, frame pointer)
x9 / s1
x10 / a0 (function argument, return value)
x11 / a1 (function argument, return value)
x12 / a2
x13 / a3
x14 / a4
x15 / a5
x16 / a6
x17 / a7
x18 / s2 (saved register)
x19 / s3
x20 / s4
x21 / s5
x22 / s6
x23 / s7
x24 / s8
x25 / s9
x26 / s10
x27 / s11
x28 / t3
x29 / t4
x30 / t5
x31 / t6
```

另外，RV32I 有程序计数器（即 PC 寄存器），该寄存器不属于通用寄存器，指向当前正在执行的指令的内存地址。

通用寄存器有如下的一些别名：zero, ra, sp, gp, tp, t0-t6, s0-s11, fp(s0), a0-a7。寄存器的别名涉及到 RISC-V 上的过程调用惯例，包括哪些寄存器用来放置参数，哪些寄存器用来放置返回值，哪些寄存器是调用者保存的，哪些寄存器是被调用者保存的等。对于底层的硬件来说，这些寄存器除了 x0 之外没有任何区别。

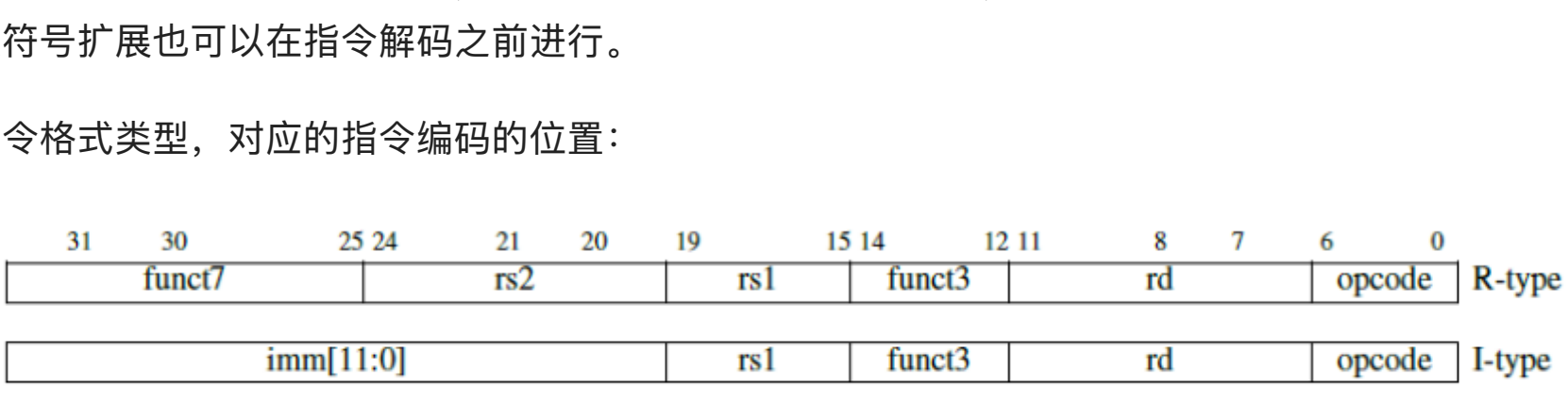
指令格式概述

RV32I 指令集体系结构有基本的 6 种指令格式，分别是：

1. 用于寄存器和寄存器之间操作的 R 类型指令；
2. 用于短立即数和访存 load 的 I 型指令；
3. 用于访存 store 的 S 型指令；
4. 用于条件跳转的 B 型指令；
5. 用于长立即数的 U 型指令；
6. 用于无条件跳转的 J 型指令。

所有指令的长度都是 32 位的。R 类型指令使用三个寄存器操作数。在指令编码上，所有指令中对应寄存器标识的位置是固定的。这样的指令格式设计使得在解码指令之前，就可以开始访问寄存器。立即数字段总是符号扩展，符号位总是在指令中的最高位，这意味着可能成为关键路径的立即数符号扩展也可以在指令解码之前进行。

指令格式类型，对应的指令编码的位置：



可以看到，B 类型和 S 类型格式，J 类型和 U 类型格式几乎是一样的，区别是对于某些字段的解释有一定的区别。所以，也可以认为 RV32I 只有 4 种类型。

另外 RV32I 也规定全 0 的指令码和全 1 的指令码都是非法指令，可以方便程序员调试。

整数运算指令

```
算术指令：add, sub
逻辑指令：and, or, xor
移位指令：sll, srl, sra
```

上述指令还有立即数的版本，立即数总是进行符号扩展。

程序可以根据比较结果生成布尔值，并将比较结果写入到目标寄存器中：slt 和 sltu。比较指令也有对应的立即数版本：slti, sltiu。（带 u 后缀的为无符号版本）

加载立即数到高位 lui 将 20 位常量加载到寄存器的高 20 位，接着可以使用标准的立即数指令来创建 32 位常量。由于 32 位常量本来就需要占用全部的 32 位，而指令长度也是 32 位，因此不得不使用拼接的方式来构造出 32 位的常量来放置到 32 位的寄存器中，使用 lui 指令来达到这个目的。

auipc 指令将立即数左移 12 位加到 PC 上，并将结果写入到目标寄存器。这样，可以将 auipc 中的 20 位立即数与 jalr 中的 12 位立即数组合，将执行流程转移到任何相对于 PC 寄存器的 32 位偏移地址。另外，auipc 指令组合普通加载或存储指令中的 12 位立即数偏移量，可以使得程序访问任何相对于 PC 寄存器的 32 位偏移地址处的数据。这些特性对于实现位置无关的代码是非常有帮助的。

访存指令

```
32bits: lw sw
16bits: lh lhu sh
8bits: lb lbu sb
```

RISC-V 是精简指令集体系结构（RISC：Reduced Instruction Set Computer），使用明确的内存访问指令来访问内存，其它的指令都不会涉及到内存。在 RV32I 中支持的寻址模式是符号扩展 12 位立即数加上基地址寄存器。另外，RISC-V 使用的是小端机结构。

条件分支指令

```
beq
bne
bge
bgeu
blt
bltu
```

条件分支指令用于程序中条件分支的执行。由于 RISC-V 指令长度必须是两个字节的倍数，分支指令确定目标地址的方式是 12 位的立即数乘以 2，进行符号扩展，然后叠加到 PC 上作为分支的跳转地址。另外，值得注意的是 RISC-V 指令中没有条件码。

大位宽数据加法

在 RV32I 中，每个寄存器是 32 位，如果要实现 64 位，就需要把 64 位整数分成高 32 位和低 32 位，保存在两个寄存器中。

那么如何实现 64 位整数求和？这种大位宽数据的加法的实现如下：

```
// {a3, a2} 即把低位保存在 a2，高位保存在 a3 的 64 位整数
// {a3, a2} + {a5, a4} = {a1, a0}
add a0, a2, a4
sltu a2, a0, a2
add a1, a3, a5
add a1, a2, a1
```

检测加法溢出

RISC-V 依赖于软件进行溢出检查。

检查无符号加法的溢出：

```
add t0, t1, t2
bltu t0, t1, overflow
```

检查有符号加法的溢出：

```
add t0, t1, t2
slti t3, t2, 0 # t3 = (t2 < 0) ? 1 : 0
slt t4, t0, t1 # t4 = (t1 + t2 < t1) ? 1 : 0
bne t3, t4, overflow
```

无条件跳转指令

- jal 将下一条指令 PC+4 的地址保存到目标寄存器中，通常是返回地址寄存器 ra。如果使用 x0 来替换 ra，则可以实现无条件跳转，因为 x0 不能被更改。
- jalr 可以调用地址是动态计算出来的函数，或者也可以实现调用返回（ra 作为源寄存器，x0 作为目标寄存器）。

杂项指令

控制状态（Control Status）寄存器的相关指令。控制状态寄存器用来进行处理器的控制，包括处理器的优先级配置，中断相关处理以及虚拟地址的相关处理。

- csrrc, csrrs, csrrw, csrrci, csrrsi, csrrwi 可以用来访问控制状态寄存器。
- ecall 指令用于向运行时环境发出调用请求，一般用于实现系统调用。
- ebreak 指令将控制转移到调试环境。
- fence 指令对外部可见的访存请求，例如设备 IO 和内存访问等进行串行化。
- fence.i 同步指令和数据流。

RV32I 函数调用惯例

函数调用的惯例包括哪些寄存器用于存放参数，哪些寄存器由调用者保存，哪些寄存器由被调用者保存，并且在返回的时候恢复等函数调用相关的寄存器等资源的使用方法。在 RV32I 的调用惯例中，临时寄存器，参数寄存器在被调用者使用之后不必保存。 寄存器别名中 s 开头的寄存器为被调用者保存的寄存器，在函数调用前后要保持不变。

RV32I 函数的入口

```
entry_label:
    addi sp, sp, -framesize
    sw ra, framesize-4(sp)
```

函数的结尾部分

```
lw ra, framesize-4(sp)
addi sp, sp, framesize
ret
```

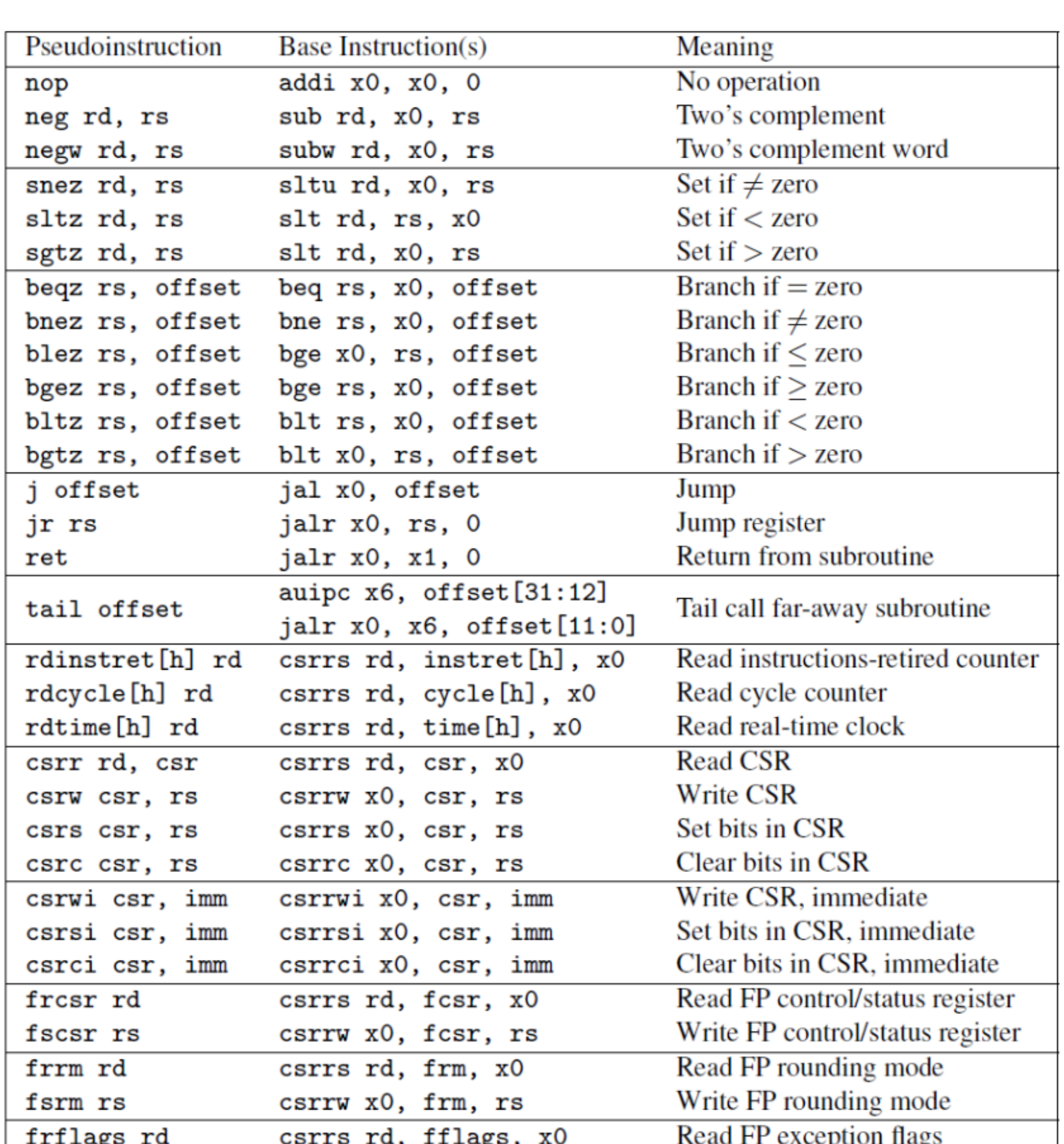
在栈空间的使用方面，进入函数的时候如果有需要的话开辟一段空间作为当前函数调用的临时空间，在函数调用返回的时候恢复栈空间。

伪指令

伪指令是为了方便程序员编写汇编代码使用的，它们不是真的机器指令，而是会被汇编器翻译为真实的物理指令。

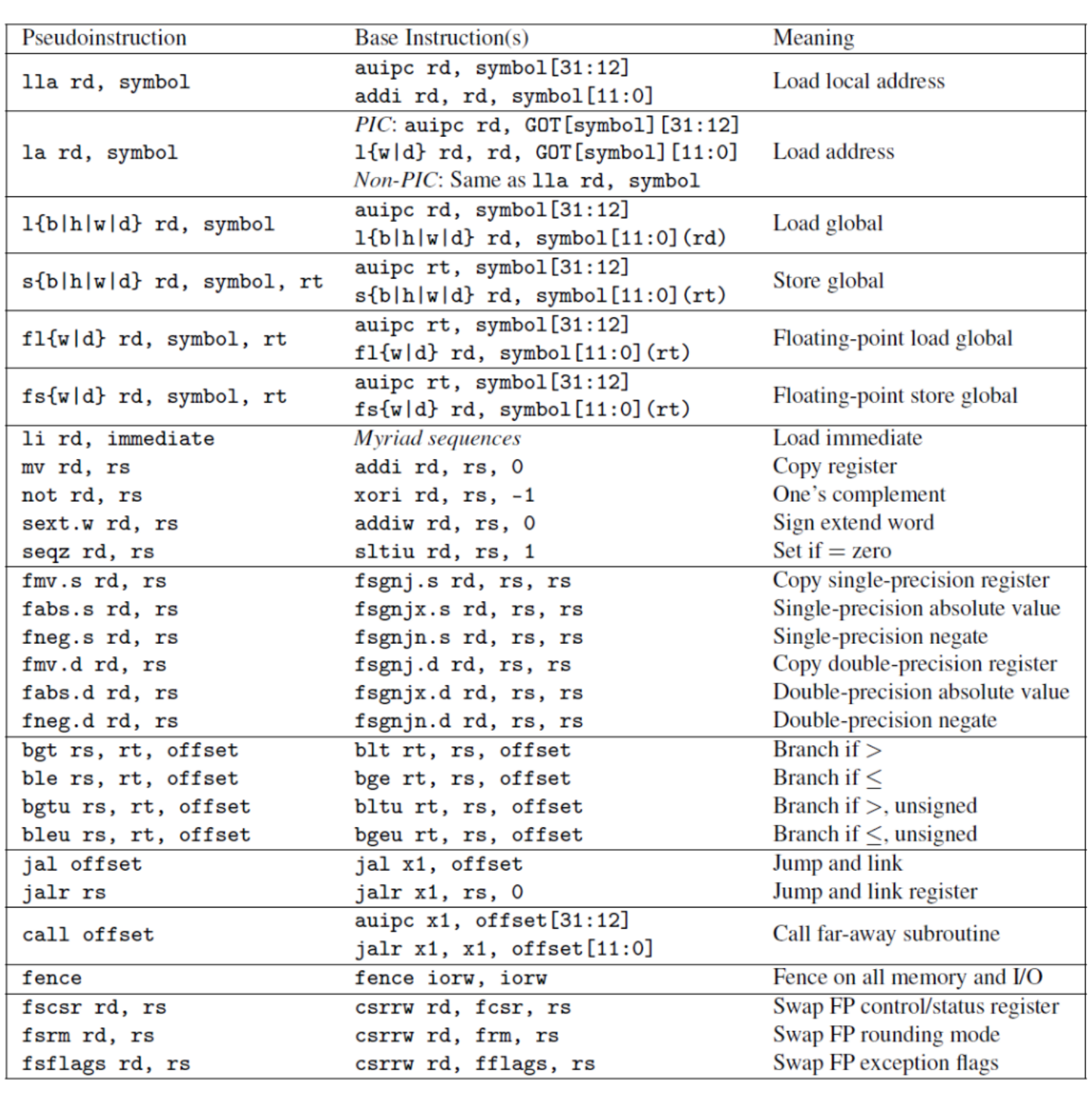
例如：ret 被翻译为：jalr x0, x1, 0

下表给出了一系列伪指令及其依赖的真实的处理器物理指令（这些伪指令都依赖于 x0 寄存器，从中可以看到 x0 寄存器的作用）。



Pseudoinstruction	Base Instruction(s)	Meaning
`nop`	`addi x0, x0, 0`	No operation
`neg rd, rs`	`sub rd, x0, rs`	Two's complement
`negw rd, rs`	`subw rd, x0, rs`	Two's complement word
`snez rd, rs`	`sltu rd, x0, rs`	Set if ≠ zero
`sltz rd, rs`	`slt rd, rs, x0`	Set if < zero
`sgtz rd, rs`	`slt rd, x0, rs`	Set if > zero
`beqz rs, offset`	`beq rs, x0, offset`	Branch if = zero
`bnez rs, offset`	`bne rs, x0, offset`	Branch if ≠ zero
`blez rs, offset`	`bge x0, rs, offset`	Branch if ≤ zero
`bgez rs, offset`	`bge rs, x0, offset`	Branch if ≥ zero
`bltz rs, offset`	`blt rs, x0, offset`	Branch if < zero
`bgtz rs, offset`	`blt x0, rs, offset`	Branch if > zero
`j offset`	`jal x0, offset`	Jump
`jr rs`	`jalr x0, rs, 0`	Jump register
`ret`	`jalr x0, x1, 0`	Return from subroutine
`tail offset`	`auipc x6, offset[31:12]` `jalr x0, x6, offset[11:0]`	Tail call far-away subroutine
`rdinstret[h] rd`	`csrrs rd, instret[h], x0`	Read instructions-retired counter
`rdcycle[h] rd`	`csrrs rd, cycle[h], x0`	Read cycle counter
`rdtime[h] rd`	`csrrs rd, time[h], x0`	Read real-time clock
`csrr rd, csr`	`csrrs rd, csr, x0`	Read CSR
`csrw csr, rs`	`csrrw x0, csr, rs`	Write CSR
`csrs csr, rs`	`csrrs x0, csr, rs`	Set bits in CSR
`csrc csr, rs`	`csrrc x0, csr, rs`	Clear bits in CSR
`csrwi csr, imm`	`csrrwi x0, csr, imm`	Write CSR, immediate
`csrsi csr, imm`	`csrrsi x0, csr, imm`	Set bits in CSR, immediate
`csrci csr, imm`	`csrrci x0, csr, imm`	Clear bits in CSR, immediate
`frcsr rd`	`csrrs rd, fcsr, x0`	Read FP control/status register
`fscsr rs`	`csrrw x0, fcsr, rs`	Write FP control/status register
`frrm rd`	`csrrs rd, frm, x0`	Read FP rounding mode
`fsrm rs`	`csrrw x0, frm, rs`	Write FP rounding mode
`frflags rd`	`csrrs rd, fflags, x0`	Read FP exception flags
`fsflags rs`	`csrrw x0, fflags, rs`	Write FP exception flags

下表是另外一些伪指令及其被翻译之后的物理指令：



Pseudoinstruction	Base Instruction(s)	Meaning
`lla rd, symbol`	`auipc rd, symbol[31:12]` `addi rd, rd, symbol[11:0]`	Load local address
`la rd, symbol`	*PIC*: `auipc rd, GOT[symbol][31:12]` `l{w|d} rd, rd, GOT[symbol][11:0]` *Non-PIC*: Same as `lla rd, symbol`	Load address
`l{b|h|w|d} rd, symbol`	`auipc rd, symbol[31:12]` `l{b|h|w|d} rd, symbol[11:0](rd)`	Load global
`s{b|h|w|d} rd, symbol, rt`	`auipc rt, symbol[31:12]` `s{b|h|w|d} rd, symbol[11:0](rt)`	Store global
`fl{w|d} rd, symbol, rt`	`auipc rt, symbol[31:12]` `fl{w|d} rd, symbol[11:0](rt)`	Floating-point load global
`fs{w|d} rd, symbol, rt`	`auipc rt, symbol[31:12]` `fs{w|d} rd, symbol[11:0](rt)`	Floating-point store global
`li rd, immediate`	*Myriad sequences*	Load immediate
`mv rd, rs`	`addi rd, rs, 0`	Copy register
`not rd, rs`	`xori rd, rs, -1`	One's complement
`sext.w rd, rs`	`addiw rd, rs, 0`	Sign extend word
`seqz rd, rs`	`sltiu rd, rs, 1`	Set if = zero
`fmv.s rd, rs`	`fsgnj.s rd, rs, rs`	Copy single-precision register
`fabs.s rd, rs`	`fsgnjx.s rd, rs, rs`	Single-precision absolute value
`fneg.s rd, rs`	`fsgnjn.s rd, rs, rs`	Single-precision negate
`fmv.d rd, rs`	`fsgnj.d rd, rs, rs`	Copy double-precision register
`fabs.d rd, rs`	`fsgnjx.d rd, rs, rs`	Double-precision absolute value
`fneg.d rd, rs`	`fsgnjn.d rd, rs, rs`	Double-precision negate
`bgt rs, rt, offset`	`blt rt, rs, offset`	Branch if >
`ble rs, rt, offset`	`bge rt, rs, offset`	Branch if ≤
`bgtu rs, rt, offset`	`bltu rt, rs, offset`	Branch if >, unsigned
`bleu rs, rt, offset`	`bgeu rt, rs, offset`	Branch if ≤, unsigned
`jal offset`	`jal x1, offset`	Jump and link
`jalr rs`	`jalr x1, rs, 0`	Jump and link register
`call offset`	`auipc x1, offset[31:12]` `jalr x1, x1, offset[11:0]`	Call far-away subroutine
`fence`	`fence iorw, iorw`	Fence on all memory and I/O
`fscsr rd, rs`	`csrrw rd, fcsr, rs`	Swap FP control/status register
`fsrm rd, rs`	`csrrw rd, frm, rs`	Swap FP rounding mode
`fsflags rd, rs`	`csrrw rd, fflags, rs`	Swap FP exception flags

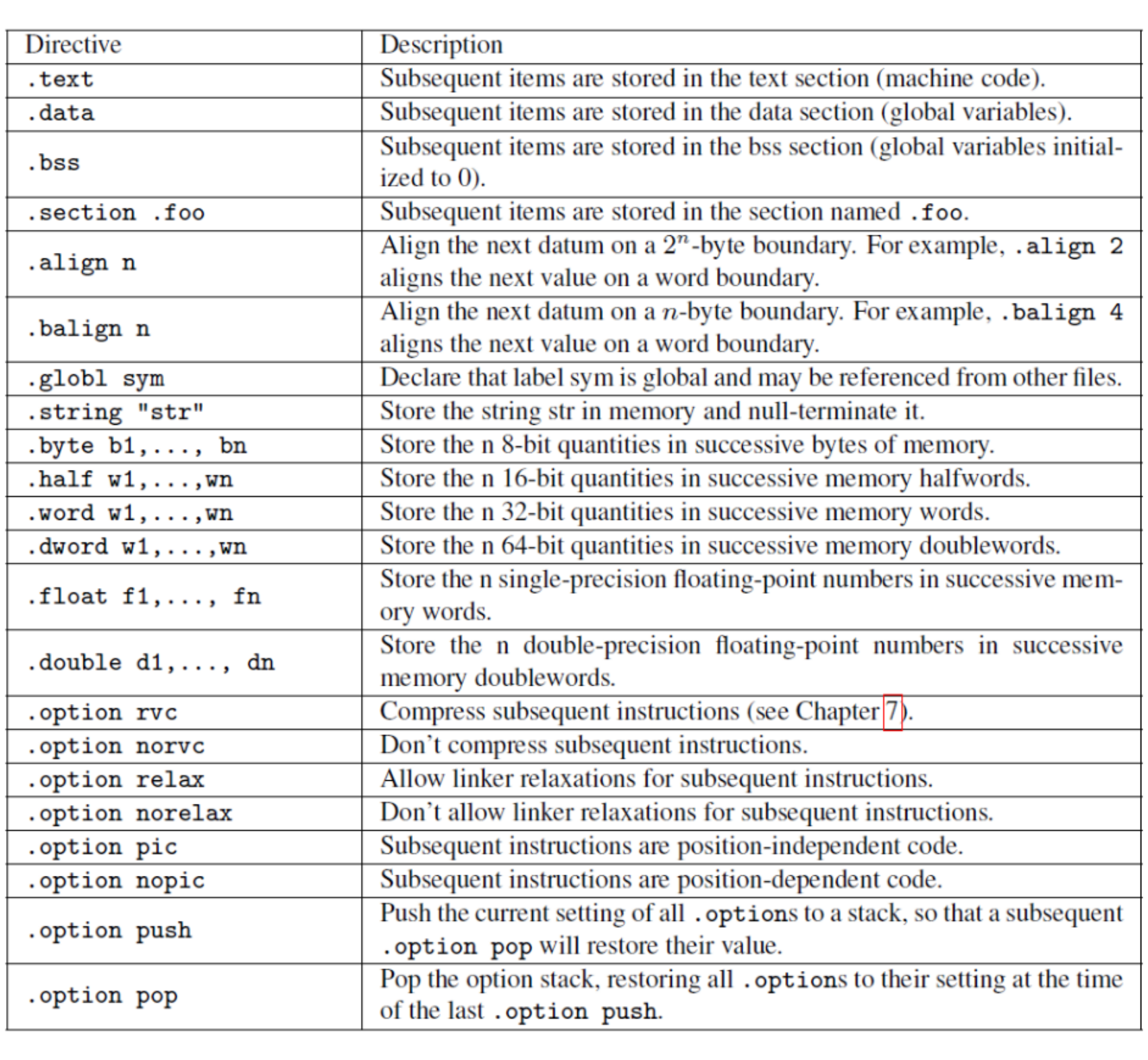
汇编器指导语句

下面是汇编器指导语句（assembler directives），可以在汇编语言的源代码中看到。了解这些指示指导语句可以帮助理解汇编语言编写的程序。

```
.text        # 进入代码段
.align 2     # 后续代码按照 2 字节对齐
.global main # 声明全局符号 main

.section .rodata        # 进入只读数据段
.balign 4               # 数据段按照 4 字节对齐
.string "hello, %s!\n"  # 创建空字符结尾的字符串
```

更多汇编器指导语句见下表：



Directive	Description
`.text`	Subsequent items are stored in the text section (machine code).
`.data`	Subsequent items are stored in the data section (global variables).
`.bss`	Subsequent items are stored in the bss section (global variables initialized to 0).
`.section .foo`	Subsequent items are stored in the section named `.foo`.
`.align n`	Align the next datum on a 2^n-byte boundary. For example, `.align 2` aligns the next value on a word boundary.
`.balign n`	Align the next datum on a n-byte boundary. For example, `.balign 4` aligns the next value on a word boundary.
`.globl sym`	Declare that label sym is global and may be referenced from other files.
`.string "str"`	Store the string str in memory and null-terminate it.
`.byte b1,..., bn`	Store the n 8-bit quantities in successive bytes of memory.
`.half w1,...,wn`	Store the n 16-bit quantities in successive memory halfwords.
`.word w1,...,wn`	Store the n 32-bit quantities in successive memory words.
`.dword w1,...,wn`	Store the n 64-bit quantities in successive memory doublewords.
`.float f1,..., fn`	Store the n single-precision floating-point numbers in successive memory words.
`.double d1,..., dn`	Store the n double-precision floating-point numbers in successive memory doublewords.
`.option rvc`	Compress subsequent instructions (see Chapter 7).
`.option norvc`	Don't compress subsequent instructions.
`.option relax`	Allow linker relaxations for subsequent instructions.
`.option norelax`	Don't allow linker relaxations for subsequent instructions.
`.option pic`	Subsequent instructions are position-independent code.
`.option nopic`	Subsequent instructions are position-dependent code.
`.option push`	Push the current setting of all `.options` to a stack, so that a subsequent `.option pop` will restore their value.
`.option pop`	Pop the option stack, restoring all `.options` to their setting at the time of the last `.option push`.

2023年9月19日 Youyou Lu, gaoyichuan, Kang Chen, Jiajie Chen, 王拓为, kusile, cuibst

作者：Youyou Lu (47.11%), gaoyichuan (22.67%), Kang Chen (16.89%), Jiajie Chen (11.11%), 王拓为 (0.89%), kusile (0.89%), cuibst (0.44%)

Copyright © 2020-2026 清华大学计算机组成原理教学组。All Rights Reserved.
Made with Material for MkDocs